The Citadel System

Project Final Delivery

Sara Gibert & Oriol Pros

sara.gibert & oriol.pros

Group 4

17/05/2026

Valéria Soldera
Operating Systems

Table of Contents

Abstract **4**
Project Design **5**
Phase 1 5
Diagrams 5
System Architecture 5
Program Execution 7
Command Processing 8
Trading System 8
Implementation Details 9
Memory Management 9
Data Structures 9
System Resources 11
Phase 2 12
Diagrams 12
System Architecture 12
Program Execution 13
Command Processing 14
Network Protocol 15
Alliance System (PLEDGE) 15
Remote Product Listing (LIST PRODUCTS <realm>) 17
Trading System 17
Data Structures 17
System Resources 18
Phase 3 19
Diagrams 19
System Architecture 19
Program Execution 21
Command Processing 21
File transfer & MD5SUM Protocol 22
Trading System 22
Data Structures 23
System Resources 24
Phase 4 26
Diagrams 26
System Architecture 26
Program Execution 27
Command Processing 28
Data Structures 29
System Resources 30
Optionals Implemented 31
Observed Problems & Solutions **32**
Phase 1 32

Phase 2....33
Phase 3....34
Phase 4....35
Temporal Estimation....37
Phase 1....37
Sara....37
Oriol....38
Phase 2....38
Sara....38
Oriol....39
Phase 3....40
Sara....40
Oriol....40
Phase 4....41
Sara....41
Oriol....42
Total Time....43
Conclusions & Proposals for Improvement....44
2025-2026 Project's Theme....46
Bibliography....47

Abstract

The Citadel System is a distributed software project that simulates a decentralized network of autonomous realms capable of forming alliances and conducting trade without any central authority. Inspired by the dynamics of a fragmented kingdom, the system is designed to explore and apply core operating systems concepts such as process management, inter-process communication (IPC), concurrency control, and network communication over sockets.

The project is structured into four progressive development phases. In Phase 1, the Maester process is implemented as the core entity of each realm, responsible for reading configuration and inventory files and providing an interactive command-line interface for managing realms and initiating trade-related operations. Phase 2 introduces socket-based communication between Maesters, enabling the establishment of alliances and the exchange of messages across a multi-hop routing network. This phase also incorporates routing logic and checksum validation to ensure reliable message delivery across intermediate nodes.

Phase 3 extends the system with file transfer capabilities, allowing the transmission of structured data such as alliance sigils and trade orders, while introducing integrity verification mechanisms to guarantee consistency between sent and received data. Finally, Phase 4 integrates concurrency through Envoy processes, enabling each Maester to delegate missions and manage multiple simultaneous operations, significantly improving scalability and responsiveness within the system.

Overall, the Citadel System demonstrates how distributed nodes can operate collaboratively in a non-centralized environment while maintaining reliability, consistency, and scalability. The project addresses fundamental challenges in distributed systems, including message routing, synchronization, resource management, and fault tolerance.

This document represents a work in progress and serves as a structured report template. It shows the evolution of the project while implementing it and it covers all implementation and analysis from Phase 1 to the Final Delivery (Phase 4).

Project Design

The Citadel System runs on a distributed architecture where every realm is its own Maester process, responsible for managing its local resources and keeping in touch with the other realms. The system is broken down into distinct modules that handle command processing, data management, and communication separately, keeping things clean, easy to scale, and straightforward to maintain.

Phase 1

Phase 1 is all about laying the groundwork and building the core structure of the system and the interactive terminal that lets the Maester process manage its realm locally. This phase covers the basics: command processing, loading configuration files, and handling inventory. The idea is to get a solid foundation in place that supports user interaction and sets the system up for the networking and distributed features that come in later phases.

Diagrams

A command-driven, modular system that mimics a Citadel trade management tool is implemented in this project. It allows a Maester to load configuration files, peruse worlds and products that are available, and initiate trading sessions where items can be added or withdrawn from a shopping list prior to the final request being sent.

Because the software is written in C and has a modular architecture, it is straightforward to comprehend, maintain, and reuse. Whether it's loading settings, processing commands, overseeing a trade session, or performing utility tasks, each module has a distinct role to play.

Block diagrams and flowcharts covering how the modules interact, how the program flows during execution, how commands are processed, and how the data structures are arranged are included in the report to make the internal workings easier to understand. The architecture and functionality of the system are explained in detail in the sections that follow.

System Architecture

The program is divided into a number of separate modules that work together to give the system's complete capability. Every module has a distinct function and uses well-defined interfaces to interact with other modules.

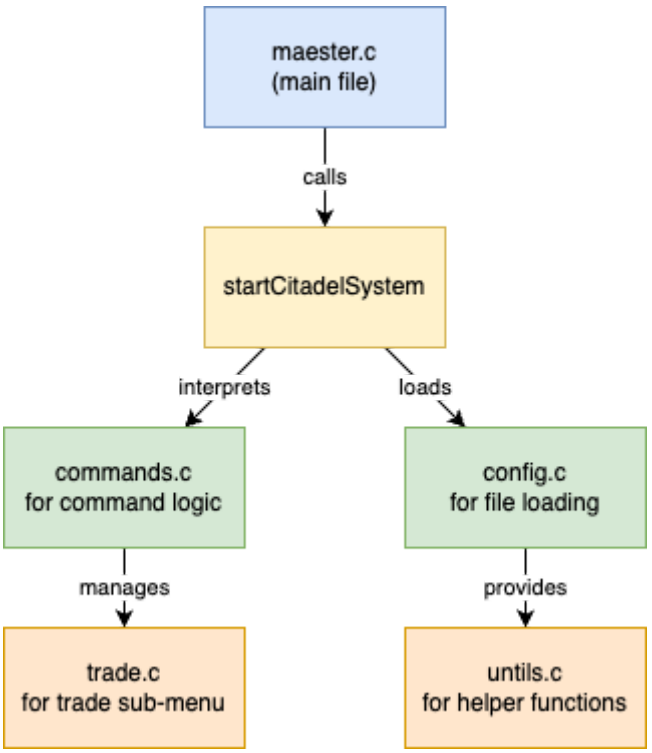


Figure 1: Diagram of System Architecture

The following primary modules make up the architecture:

- The main module, maester.c:

The program's entry point is this module. It starts the main execution loop, installs signal handlers, reads the configuration files, and initializes the system. Instead of implementing business logic directly, the main module facilitates communication between the other modules.

- configuration module

This module is in charge of loading and maintaining the configuration data that the system needs. It reads both the stock file, which contains product details, and the master file, which contains details about accessible worlds and routes. Additionally, when the program is finished, the module offers routines for freeing the memory that was allocated.

- module for commands

User commands entered in the Citadel console are interpreted by the commands module. It chooses the operation to perform after obtaining a parsed list of words. It calls functions that list products, list kingdoms, or initiate a trading session, depending on the command.

- module for trading

Every aspect of trading sessions is managed by this module. After a transaction is started, it controls a secondary command loop in which the user can submit the completed list to the chosen realm, add items to a shopping list, or remove them.

- utils module

Helper functions used throughout the program are contained in the utility module. These consist of memory management assistants, string parsing tools, and input/output assistants. Reuse is enhanced and code duplication is prevented by centralizing these functionalities.

Program Execution

The program is executed in a predetermined order that begins with startup and ends with a command-processing loop.

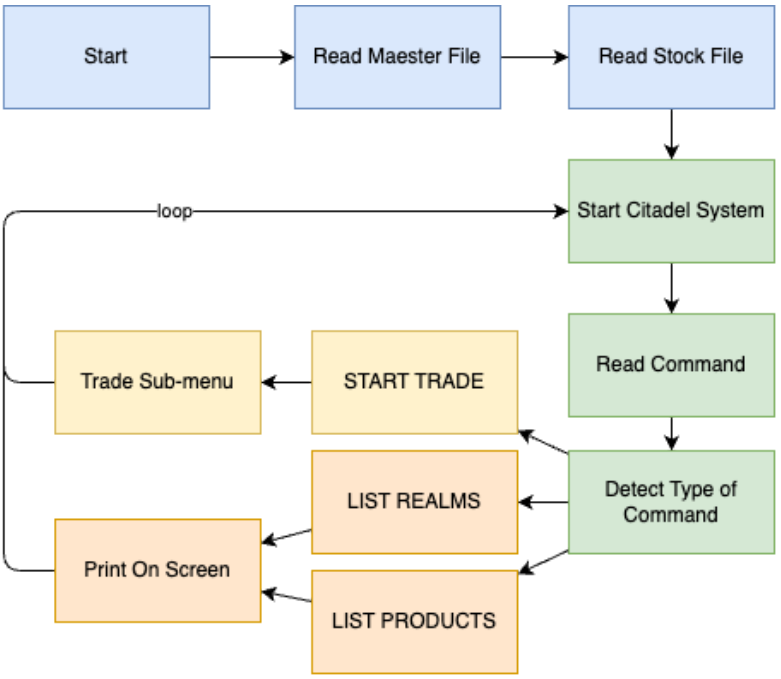


Figure 2: Diagram of Program Execution

When the software is run from the command line, the process starts. Before starting the main interaction loop, the main function completes a number of initialization procedures. Initially, the configuration module's functions are used by the system to read the configuration files. These files include details regarding the things that can be traded and the realms that are available.

Then, the program calls the function that initiates the Citadel system after loading the configuration data. At this point, the program goes into an interactive loop where it waits for commands from the user continuously. The program uses the command detection technique to parse a line it reads from standard input inside this loop. The related function is carried out based on the command that the user enters. For Phase 1, the user can ask to begin a trade session, list every realm, or list every product that is accessible.

Until the user exits the program or a termination signal is received, this loop keeps going. To guarantee appropriate resource management, the software releases all dynamically created memory before ending.

Command Processing

All user interactions are managed by a command-processing procedure once the system is running.

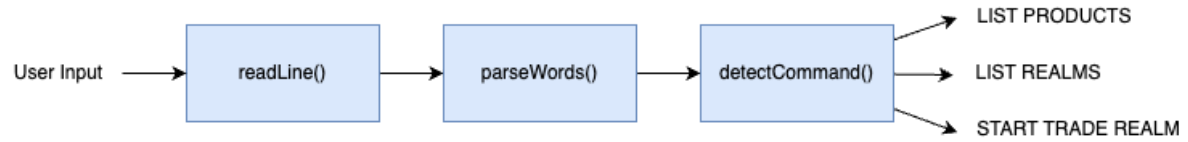


Figure 3: Diagram of Command Processing

A multi-stage function is the design of the command processing mechanism. The user's command is first read by the program as a raw text line. After that, a parsing function receives this input and divides the line into distinct words. This procedure transforms the input into a structured representation that makes analysis by the computer simpler.

Afterwards, the list of words is supplied to the command detection function once the input has been tokenized. This function decides which operation should be carried out after analyzing the command. For instance:

- The relevant listing function is called if the command relates to listing products.
- The system prints the relevant data if the command asks for the list of realms that are available.
- The software switches to the trade module if the command initiates a trading session.

Trading System

When a user starts a trade with a certain realm, the trading system is called. The trade module is solely responsible for this portion of the program.

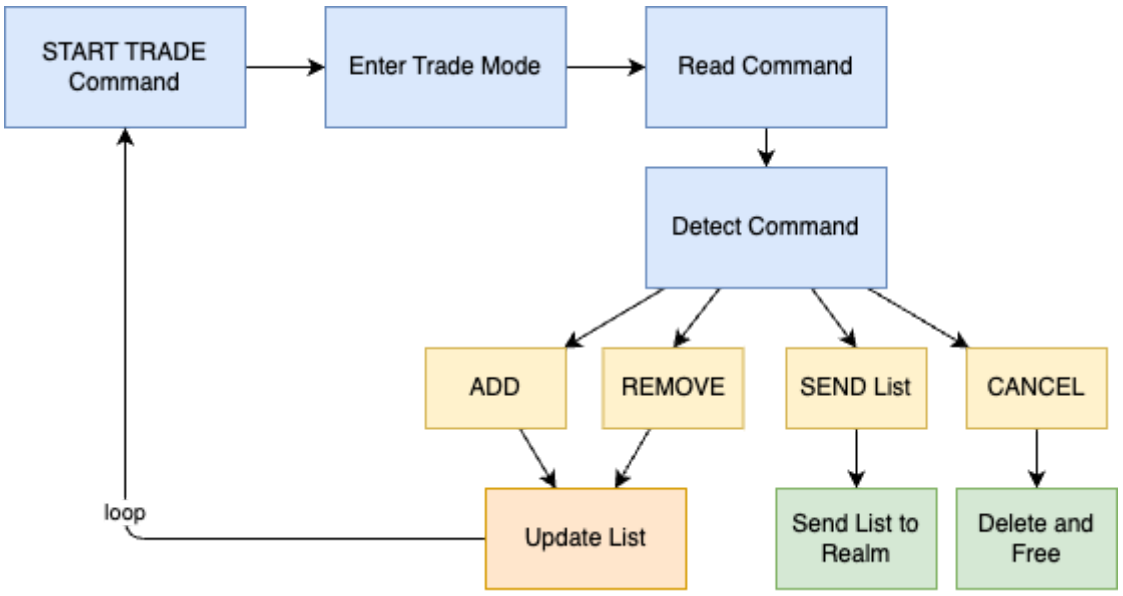


Figure 4: Diagram of Trade Sub-menu

The program switches to a certain trade mode when a trade session starts. Also, a shopping list that will eventually be transmitted to the chosen realm can be managed by the user in this manner.

Essentially, this sub-menu is similar to the main Citadel loop but restricted to trade-related activities, since the trade session runs in its own command loop. In this stage, the user can carry out commands like:

- Adding items on the shopping list
- Removing items from the list
- Sending the completed list

A specific trade command detection function interprets each command, same as the “original” terminal. Until the user chooses to send the list or end the trade session, the system will be updating the shopping list accordingly.

Implementation Details

Each source file in the implementation's modular design corresponds to a distinct subsystem. In order to guarantee that functions and data structures can be accessed without disclosing internal implementation details, header files specify the interfaces between modules. Important elements of implementation consist of:

- distinction between runtime logic and configuration loading
- unambiguous command dispatching features
- modular trade management
- shared utility functions for shared tasks

Internal documentation on the significance of key functions and parameters is also included in the code.

Memory Management

Because the program is implemented in C, memory management must be handled manually. Thus, product lists, shopping lists, and configuration data are all stored using dynamic memory allocation. The program allocates memory to hold the relevant structures when it loads configuration files. It does so by using specialized functions that release all allocated memory prior to termination in order to prevent memory leaks. When the program ends or certain resources are no longer needed, these functions are called.

Additionally, signal handling is used to prevent unanticipated termination events from preventing the release of allotted resources. The input stream is closed and the main loop gracefully ends when the program receives an interrupt signal, enabling the cleanup routines to run. Like so, it is ensured that the system ends securely and that all resources are released appropriately.

Data Structures

Several data structures were created and used in order to keep the information organized and clean, especially in this phase, which had a lot to do with managing structures. Here are the following structures that we used in order to implement correctly and efficiently Phase 1:

- The **Maester** structure represents the main entity in the system. Essentially, a realm in the network. It bundles together all the configuration data the program needs to run: the realm's

name, its communication address (IP and port), the number of envoys, and the available routes (which is another data structure). Grouping all of this into a single structure made sense because it keeps everything in one place, makes the code easier to navigate, and lets the program read and update the realm's state without jumping around.

- The **Route** structure represents a possible path to reach another realm. It holds the destination name, IP address, and port number. The idea was to treat what we thought is a “network connection” as a proper, reusable data type rather than scattering that information around. Since routes are stored as an array inside the Maester structure, the system can easily manage multiple destinations and loop through them whenever it needs to send a message or figure out how to route one in later phases.

- The **Product** structure handles the realm's inventory, which is one of the *stock.db* binary files provided to us in the statement . It stores the product name, how much of it is available, and its weight. We believe that keeping all of a product's attributes together in one structure will make future inventory operations (such as trading products), besides displaying a list of products, much cleaner and easier to work with.

- The **ShoppingList** structure represents the items being requested during a trade. It stores the product name and the amount being asked for. This was introduced specifically to keep trade requests separate from the internal inventory, since it is different from the Product Structure, which shows the available inventory of a Realm. The Shopping List structure, which shows the products willing to exchange on an ongoing trade, has a dedicated structure for requested items, makes trade operations easier to process and keeps a clear line between what a realm has and what it's asking another realm for. It is also important to mention that this structure will always be filled with inventory from the *stock.db* files provided to us, since we didn’t create one from scratch by ourselves, we simply reused one.

The following diagram has been made in order to provide a cleaner vision of how the structures interact with each other:

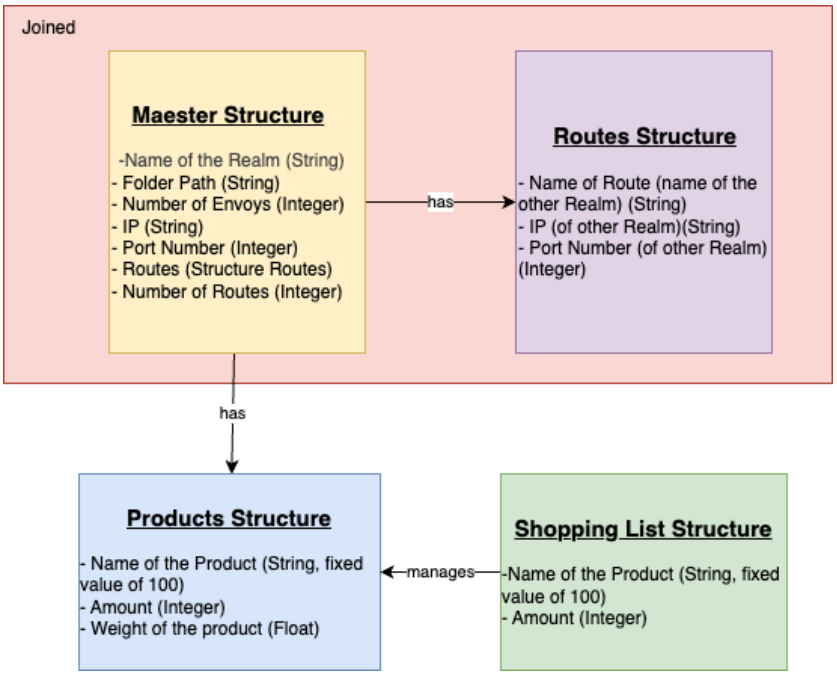


Figure 5: Diagram of Structures

System Resources

During Phase 1, the system operates using two primary OS resources: signals and file descriptors. File descriptors manage reading from and writing to configuration and database files, providing the program with a direct connection to the operating system for all necessary input and output. Regarding signals, the program only monitors SIGINT, as requested, which is the interrupt signal activated when the user presses Ctrl+C. Instead of simply terminating abruptly, the system intercepts this signal and powers down in an orderly manner, ensuring all resources are correctly freed before it exits.

Anything more than that, sockets, pipes, semaphores, and also additional Optional Implementations, does not belong to this stage. These will become relevant later when the project progresses to inter-process communication and concurrency

Phase 2

Phase 2 builds upon the solid foundation established in Phase 1 by introducing socket-based communication between Maester processes. In this phase, realms can reach out to one another over the network to negotiate alliances, query remote product inventories, and send trade requests. The terminal interface from Phase 1 is preserved and extended with new commands, while a concurrent server thread runs in the background at all times, ready to receive incoming connections from other Maesters.

Diagrams

The system grows significantly in complexity during Phase 2. The codebase is restructured into a deeper directory hierarchy that separates networking concerns from local logic, and the number of modules increases to accommodate the new functionality. The following subsections describe each design aspect in detail.

System Architecture

The modular architecture introduced in Phase 1 is preserved and expanded. Each pre-existing module retains its role, and several new ones are added to handle network operations.

The new and updated modules are:

- The main module, maester.c:

The entry point is updated to start a listening socket before entering the interactive terminal loop. The listening file descriptor is passed to a dedicated server thread via a ServerComm structure. If the socket cannot be bound, the program exits immediately. On shutdown, the listening socket is closed and the server thread is signalled and joined before memory is freed.

- The localConfig module (CONFIG/):

Inherits its responsibilities from Phase 1's config module. It reads the .dat configuration file and the binary stock database, populates the Maester structure, and provides cleanup functions. An Alliance array is now managed alongside the Route array, and cleanup functions have been updated to free the new fields.

- The networkConfig module (CONFIG/):

A new module responsible for all connection-level operations. It provides start_listen to bind and listen on a TCP socket, accept_connection to accept an incoming client, and connect_to_maester to open an outgoing connection to a remote realm. It also defines server_thread, the function that runs the background server loop.

- The frame module (SOCKET/):

A new module that defines and enforces the fixed-size 320-byte binary protocol used for all inter-Maester communication. It provides functions to build frames, send and receive them reliably over a socket, and compute and validate checksums.

- The threadHandler module (SOCKET/):

Handles the logic for each incoming frame type received by the server thread. Functions include pledge_request, pledge_response, list_product_request, and list_product_response. This module is responsible for interpreting the content of a frame and taking the appropriate action, including forwarding the frame in the case of a multi-hop route.

- The pledgeComm module (COMMANDS/PLEDGE/):

Implements the three user-facing pledge commands: PLEDGE <realm> <sigil> to send an alliance request, PLEDGE RESPOND <realm> ACCEPT/REJECT to answer a pending request, and PLEDGE STATUS to display the current state of all known alliances.

- The listComm module (COMMANDS/LIST/):

Extends the Phase 1 listing functionality. In addition to LIST REALMS and LIST PRODUCTS, it now supports LIST PRODUCTS <realm> which sends a TYPE_PROD_LIST_REQ frame to a remote ally and waits for its product inventory in the response.

- The detectCommand module (COMMANDS/):

Updated to route the new PLEDGE and extended LIST commands to their respective handlers.

Program Execution

The program startup sequence gains one new step compared to Phase 1: after reading the configuration files, the program binds and starts listening on the socket defined in the .dat file before entering the interactive loop. A thread is then created to run *server_thread* in the background.

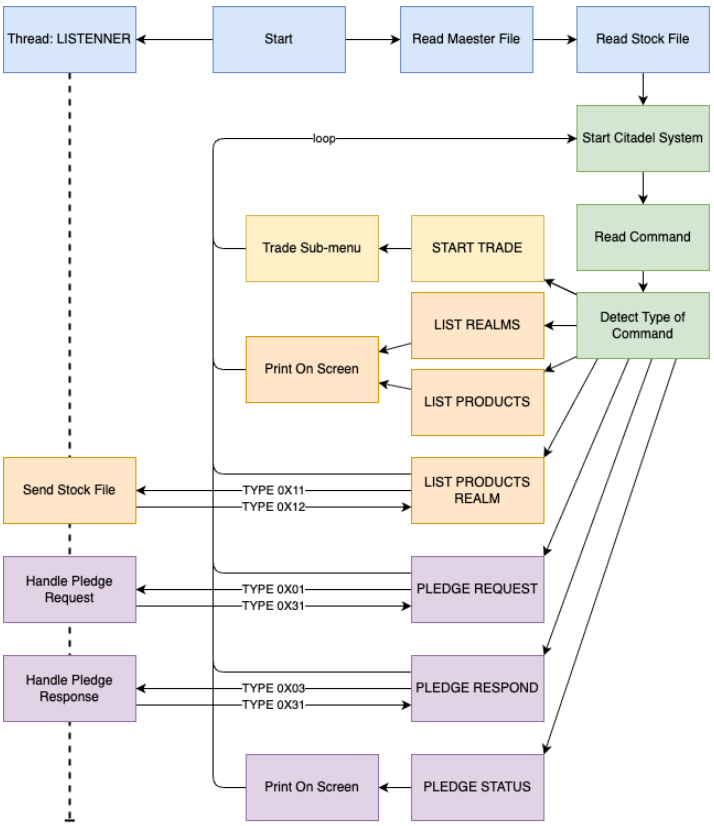


Figure 6: Diagram of Program Execution

The interactive terminal loop from Phase 1 is unchanged in its structure. It reads a command, parses it, and dispatches it through detectCommand. When the user exits or a SIGINT is received, close(0) is called to end the input loop, the listening socket is closed, and pthread_kill followed by pthread_join ensures the server thread terminates cleanly before memory is freed.

Command Processing

Command processing follows the same multi-stage pipeline as Phase 1, with the addition of two new command families.

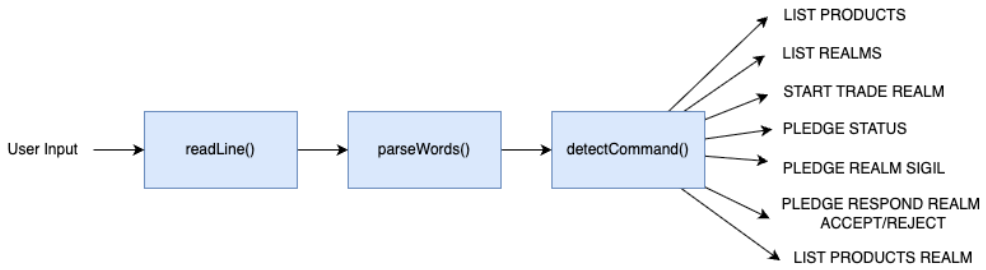


Figure 7: Diagram of Command Processing

The PLEDGE section covers alliance management. The LIST family is extended to handle the three-word LIST PRODUCTS <realm> variant, which requires network communication. The START TRADE command is preserved from Phase 1. Unknown or malformed commands still produce informative error messages.

Network Protocol

All communication between Maester processes uses a fixed binary frame of exactly 320 bytes. Every message, regardless of type, is sent and received as a complete 320-byte unit. The frame structure is as follows:

- type (1 byte): identifies the message kind (e.g. 0x01 for an alliance request, 0x03 for a response, 0x11 for a product list request, 0x69 for NACK).
- origin (20 bytes): null-padded string identifying the sender, formatted as IP:port.
- dest (20 bytes): null-padded string identifying the intended destination realm.
- data_len (2 bytes): length of the payload in network byte order.
- data (275 bytes): the payload, zero-padded to fill the field.
- checksum (2 bytes): a 16-bit checksum computed as the sum of all preceding bytes modulo 65536, stored in network byte order.

On reception, the server thread validates the checksum before dispatching the frame. If the checksum fails, a NACK frame is returned and the connection is closed. All multi-byte integer fields are converted between host and network byte order using htons/ntohs.

Alliance System (PLEDGE)

The alliance system is the central new feature of Phase 2. It allows two Maester processes to establish a mutual agreement that is required before trade can take place.

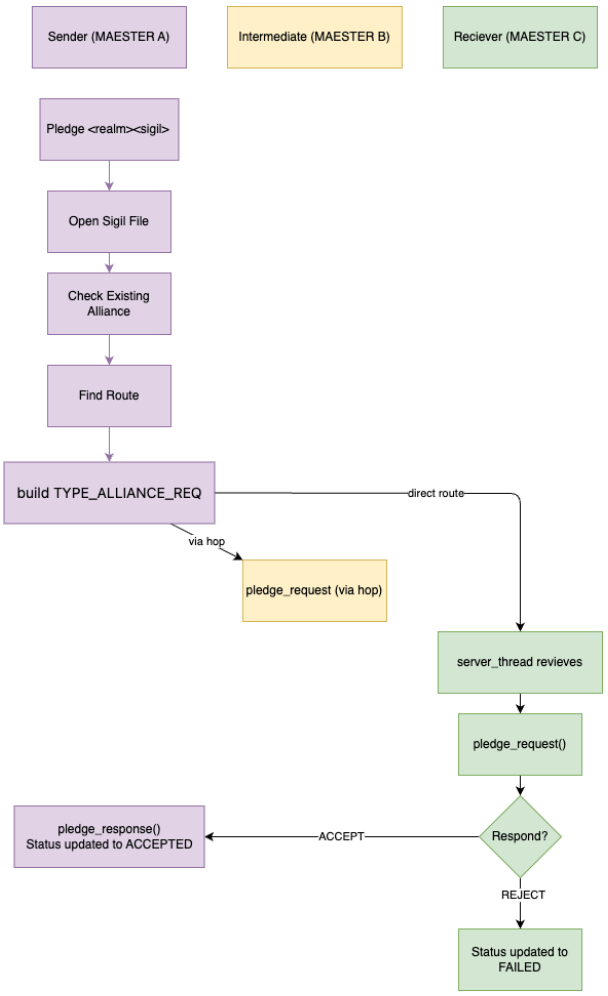


Figure 8: Diagram of Pledge Logic

When a user runs PLEDGE <realm> <sigil>, the program reads the sigil file from the REALMS/ directory, builds a TYPE_ALLIANCE_REQ frame with the sender's name, the sigil path, its size in bytes, and a placeholder MD5 checksum (to be replaced in Phase 3, for now it's hardcoded), and sends it to the destination's IP and port. The alliance is stored locally with status ALLIANCE_PENDING.

When the server thread receives a TYPE_ALLIANCE_REQ, it registers the requesting realm as a pending ally and sends back a TYPE_ACK_FILE to acknowledge receipt. The human operator at the receiving end then decides whether to run PLEDGE RESPOND <realm> ACCEPT or PLEDGE RESPOND <realm> REJECT, which sends a TYPE_ALLIANCE_RESP frame back to the original requester and updates the local alliance status accordingly.

If a route to the destination is not directly known (IP is *.*.*.*), the frame is forwarded through the DEFAULT route, implementing single-hop routing. The threadHandler module performs the same lookup when it receives a frame destined for a different realm, forwarding it transparently.
The PLEDGE STATUS command iterates over the local alliance array and prints each ally alongside its current state: PENDING, ALLIED, or FAILED.

Remote Product Listing (LIST PRODUCTS <realm>)

Once an alliance has been established, the user can query the product inventory of a remote ally with LIST PRODUCTS <realm>. The local Maester sends a TYPE_PROD_LIST_REQ frame to the ally. The server thread on the receiving end handles this by reading its own stock database and sending back the data in a TYPE_PROD_LIST_RESP frame. The requesting side then formats and displays the result.

Trading System

The trading system introduced in Phase 1 is preserved in full. The START TRADE <realm> command opens the trade sub-menu where items can be added to or removed from a shopping list, which can then be submitted or cancelled. In Phase 2 the trade networking frames (TYPE_ORDER_REQ, TYPE_ORDER_DATA, TYPE_ORDER_RESP) are defined in the protocol but their full network-level implementation is deferred to a later phase.

Data Structures

The data structures from Phase 1 are preserved and two new ones are added:

- The Alliance structure represents a known alliance relationship with another realm. It stores the realm name as a string, its IP address, its port number, a status integer (PENDING = 0, ACCEPTED = 1, REJECTED = 2), and a timestamp recording when the alliance was initiated, which will be used for timeout handling in later phases. The Alliance array lives inside the Maester structure and grows dynamically as new alliances are created.

- The Frame structure is a 320-byte struct that represents a single network message. The __attribute__((packed)) annotation ensures no compiler padding is inserted, so the in-memory layout maps exactly to what is sent over the wire. This allows frames to be sent and received with a simple write/read loop without any serialisation step.

- The ServerComm structure is a small helper used to pass arguments to the server thread at creation time. It bundles the listening file descriptor and a pointer to the Maester structure, since pthread_create accepts only a single void* argument.

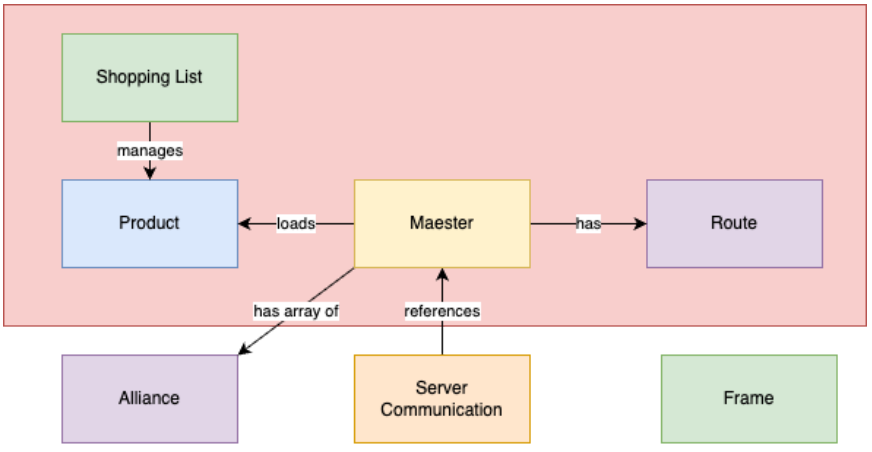


Figure 9: Diagram of Data Structures

System Resources

Phase 2 introduces three new OS resource categories on top of the signals and file descriptors used in Phase 1:

- Sockets: Each Maester binds one TCP server socket at startup using socket, bind, and listen. For every outgoing connection (pledge request, pledge response, product list request), a separate client socket is created with connect_to_maester and closed immediately after the transaction. The server socket is kept open for the lifetime of the process.

- Threads: A single thread (pthread_t t_server) is created at the start and runs server_thread in parallel with the interactive terminal. Coordination between the main thread and the server thread is minimal by design: the main thread signals the server thread with SIGINT when it is time to exit, and then joins it before freeing memory.

- Network byte order conversions: All multi-byte values placed in frames (data_len, checksum) are converted to network byte order with htons before transmission and back with ntohs on reception, ensuring correctness across machines with different endianness.

Phase 3

Phase 3 builds upon the networked foundation established in Phase 2 by introducing full file transfer capabilities between Maester processes. In this phase, realms are no longer limited to exchanging control messages, but can now transmit sigils, product lists, and trade orders as complete files across the network. Alliances are further reinforced through verified sigil exchanges, while trade requests evolve into actual data-driven operations involving inventory files. Also, the terminal interface from previous phases is the same and further extended to support these interactions, while the concurrent server thread continues to run in the background, now handling both message-based communication and fragmented data transfers from other Maesters.

Diagrams

The system structure introduced in Phase 2 is preserved in its entirety. The changes in Phase 3 are concentrated in the trading pipeline: the tradeFunc module, which previously only managed the local shopping list, now handles the complete order request protocol, and the listComm module, which previously only retrieved the remote product list for display, now stores it inside the Alliance struct so that it can be reused during a trade session. A still unused envoyComm module is also introduced in this phase, reserved for the Envoy functions and features of Phase 4.

System Architecture

The modular directory hierarchy from Phase 2 is unchanged. The additions and updates are:

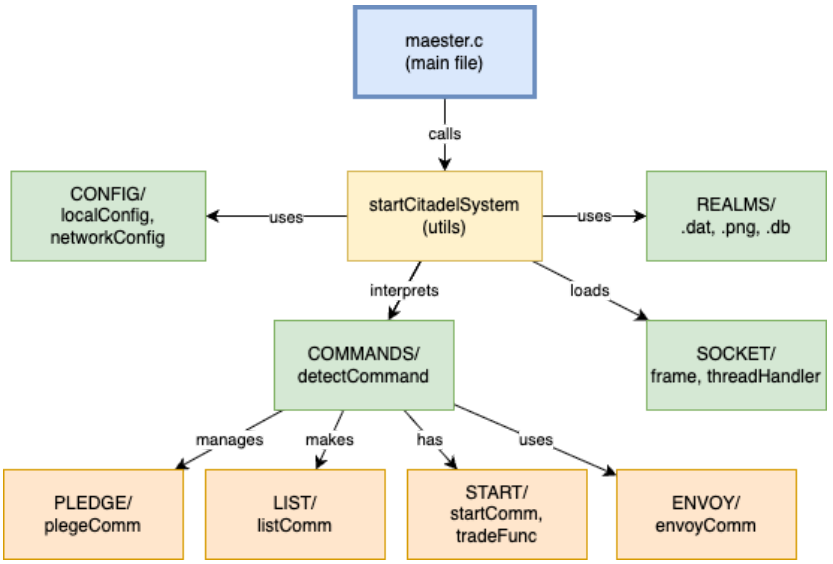


Figure 10: Diagram of System Architecture

- The tradeFunc module (COMMANDS/START/):

The most significantly extended module in Phase 3. Two new functions are added: sendShoppingList(), which connects to the ally, sends a TYPE_ORDER_REQ header frame containing the filename, file size and MD5 checksum of the shopping list file, then streams the file contents as TYPE_ORDER_DATA chunk frames and waits for a TYPE_ACK_MD5 and a final

TYPE_ORDER_RESP, and updateStockFromOrder(), which runs on the receiver side, validates that every requested product exists and is sufficiently stocked, and then atomically updates the binary .db stock file by writing a corrected copy to a temporary file and renaming it into place.

- The threadHandler module (SOCKET/):

Extended with a new handler, order_request(), which receives an incoming TYPE_ORDER_REQ frame, acknowledges it with TYPE_ACK_FILE, receives the shopping list file in chunks, computes its MD5, returns a TYPE_ACK_MD5, calls updateStockFromOrder(), and finally sends a TYPE_ORDER_RESP indicating either success or the reason for rejection. The existing pledge_request() handler is also updated: the hardcoded MD5 placeholder from Phase 2 is replaced with a real call to calculate_md5sum(), and the TYPE_ACK_MD5 exchange that was absent in Phase 2 is now fully implemented on both sides. A new function, disconnect_maester(), handles incoming TYPE_DISCONNECT frames and updates the corresponding alliance status to ALLIANCE_DISCONNECTED.

- The frame module (SOCKET/):

Gains calculate_md5sum(), which forks a child process, redirects its standard output through a pipe, and calls execv("/usr/bin/md5sum", ...) to compute the 32-character hexadecimal digest of any file. The parent reads exactly 32 bytes from the pipe and returns the result as a heap-allocated string. Also, it is important to mention that we firstly tried to leave the path for the *execv* as md5sum but didn't recognize the command. Thus, we ended up using the path shown previously.

- The networkConfig module (CONFIG/):

Extended with maesterDisconnection(), which iterates over all accepted alliances at shutdown and sends a TYPE_DISCONNECT frame to each one, so that allied Maesters are notified when a peer goes offline. This was supposed to be implemented in phase 2, but we didn't succeed at that moment. Now it is well implemented with this function call.

- The listComm module (COMMANDS/LIST/):

listProductsRealm() is updated to store the received products directly into the Alliance struct's new products array, in addition to printing them. This cached inventory is used by startTradeRealm() to populate the trade sub-menu.

- The startComm module (COMMANDS/START/):

startTradeRealm() now checks that the ally's product list has been loaded before entering the trade sub-menu, and displays the available products to the user at the start of a trade session.

- The envoyComm module (COMMANDS/ENVOY/):

A new unused module introduced in Phase 3 to organize the groundwork for Phase 4. The module compiles and links cleanly but does not yet implement any functionality.

Program Execution

The program startup sequence is identical to Phase 2. No changes are made to maester.c itself: the listening socket is still bound before the interactive loop, the server thread is still launched via pthread_create, and shutdown still closes the listening socket, signals the thread, and joins it before freeing memory. The only difference visible at startup is that the program now also calls maesterDisconnection() just before freeMaester(), notifying all active allies that the process is ending, and the 2 new modules implemented using the protocols described in the statement. Here is an updated version for the diagram of the program execution.

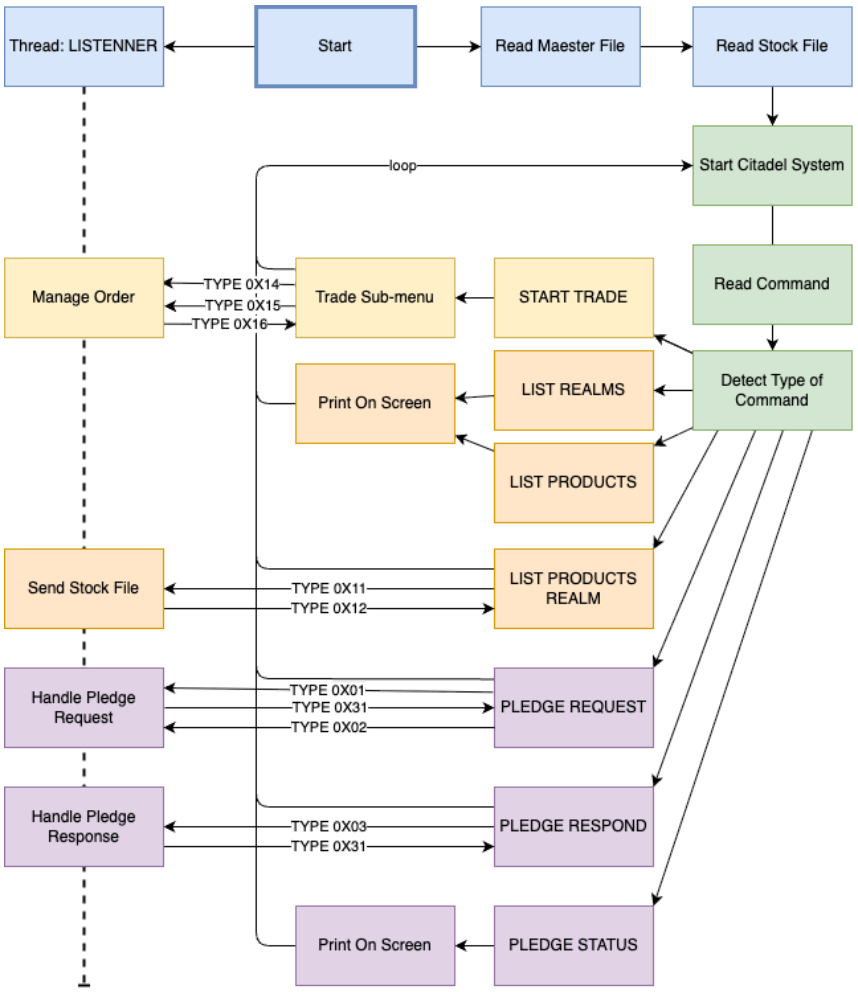


Figure 11: Diagram of Program Execution

Command Processing

Command processing follows the same multi-stage pipeline as Phase 2 without structural changes. The detectCommand() dispatcher is not modified. The difference in Phase 3 is entirely in what happens downstream: START TRADE <realm> now triggers a full file transfer exchange rather than a local-only operation, and LIST PRODUCTS <realm> now caches the received product array into the alliance in addition to displaying it. These functionalities are already displayed in the diagram for phase 2.

File transfer & MD5SUM Protocol

Phase 3 introduces a reusable file transfer sub-protocol that is used consistently for three different operations: the sigil image transfer during a PLEDGE request, the stock database transfer during LIST PRODUCTS <realm>, and the shopping list transfer during START TRADE <realm>. In all three cases the pattern is identical: the sender first transmits a header frame containing the filename, the total file size in bytes, and the MD5 checksum of the file, then the receiver acknowledges with a TYPE_ACK_FILE frame, the sender then streams the file content in TYPE_ORDER_DATA (or TYPE_SIGIL_DATA / TYPE_PROD_LIST_DATA) chunk frames of up to 275 bytes each, which is the maximum payload size of the 320-byte protocol frame. Once the receiver has accumulated the expected number of bytes, it independently computes the MD5 of the received data using calculate_md5sum() and returns a TYPE_ACK_MD5 frame containing either CHECK_OK or CHECK_KO and the receiver's realm name, and if the check fails, the operation is aborted and any cached data is discarded.

The MD5 computation itself is an important OS-level feature of this phase. Rather than implementing MD5 natively in C, the system delegates it to the standard md5sum utility using fork() and execv(). A pipe is created before the fork, then the child process redirects its standard output to the write end of the pipe and calls execv("/usr/bin/md5sum", args) with the target file path, the parent reads exactly 32 bytes from the read end of the pipe, null-terminates them, and returns the digest. This approach introduces fork() as a genuine OS resource in Phase 3, used specifically for the integrity check, independent of the Envoy concurrency that is planned for Phase 4.

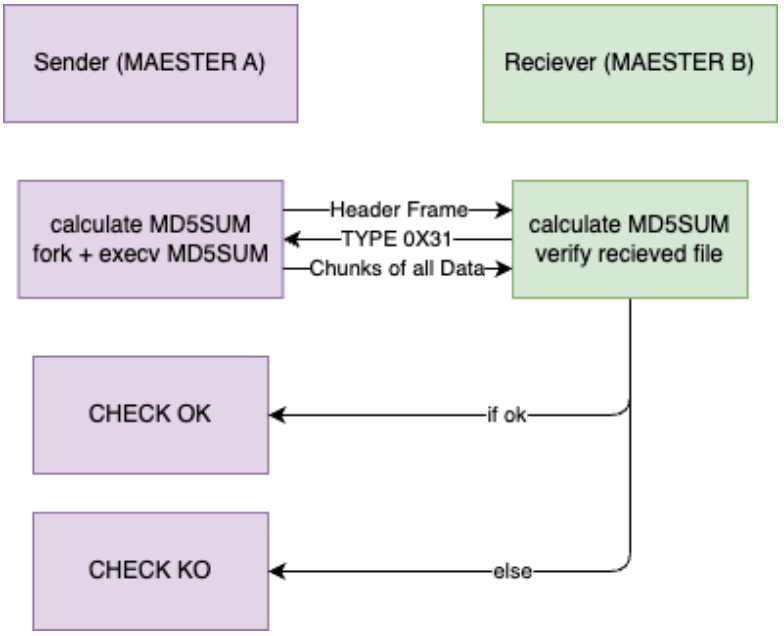


Figure 12: Diagram of MD5SUM Protocol

Trading System

The trading system is now fully implemented end-to-end. The START TRADE <realm> command first verifies that the target realm is an active ally and that its product list has been cached via LIST PRODUCTS <realm>, and if either condition is not met, the command prints an error and returns to the main loop. Once inside the trade sub-menu, the user sees the ally's available products and can add

or remove items from the shopping list using the same add <product> <quantity> and remove <product> <quantity> commands from Phase 1.

When the user types SEND, the system calls makeShoppingFile() to serialise the shopping list into REALMS/shopping_list.txt in the format <product>&<quantity>\n, then calls sendShoppingList() to transfer that file to the ally using the file transfer sub-protocol described above. On the receiver's side, order_request() receives the file, verifies its integrity, and calls updateStockFromOrder(). This function makes two passes over the order file: the first pass validates that every requested product exists in the local stock database and that the requested quantity does not exceed what is available, returning a rejection reason such as OUT_OF_STOCK or UNKNOWN_PRODUCT if any check fails, the second pass, executed only if the first passes without error, reads the stock binary file, subtracts the ordered quantities, writes the corrected records to a temporary file, and renames it over the original. The receiver then sends a TYPE_ORDER_RESP frame containing either OK or REJECT&<reason>. If the sender receives OK, it also updates its local cached view of the ally's inventory by subtracting the purchased quantities from the Alliance.products array.

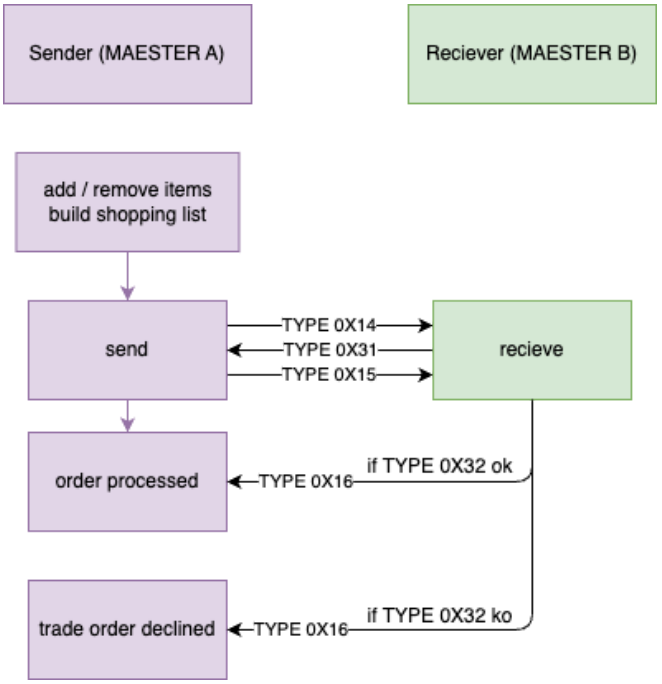


Figure 13: Diagram of Trading System & Order Transfer

Data Structures

The data structures from Phase 2 are still preserved. Two fields are added to existing structures:

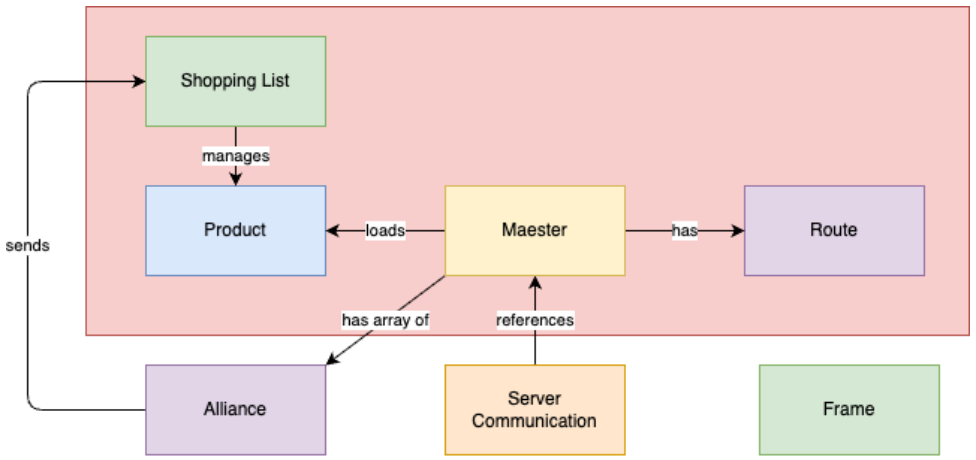


Figure 14: Diagram of Data Structures

- The Alliance structure gains two new fields: products, a dynamically allocated array of Product structs, and numProducts, an integer tracking how many entries the array holds. These fields are populated when the user runs LIST PRODUCTS <realm> and the received stock data passes the MD5 integrity check. They are freed in freeMaester() alongside the rest of the alliance data. Having the ally's product list stored inside the Alliance struct rather than in a separate global data structure makes it straightforward to look up what a specific ally stocks without additional indirection, and ensures the data is cleaned up automatically when the alliance is freed.

- The Maester structure gains one new field: stockName, a string holding the path to the local stock database file as passed on the command line. Storing this path in the Maester struct means that any module needing to read or update the local stock, in particular list_product_request() in the server thread and updateStockFromOrder() on the receiver side, can access it through the Maester pointer without requiring the path to be passed as a separate argument through every function call.

System Resources

Phase 3 introduces two new OS resource categories on top of those used in Phase 2:

- **Pipes and fork():** Every call to calculate_md5sum() creates a pipe using pipe(), forks a child process with fork(), redirects the child's standard output to the write end of the pipe using dup2(), and executes md5sum via execv(). The parent reads the result from the read end of the pipe and closes both file descriptors after use. This means that for every file transfer in Phase 3 (sigil, stock database, or shopping list), at least one child process is created and two additional file descriptors are opened and closed. Proper cleanup is ensured by always closing both pipe ends in both parent and child, even in error paths.

- **Disconnection notifications:** At shutdown, maesterDisconnection() iterates over the alliance array and opens a new outgoing TCP socket to each active ally, sending a TYPE_DISCONNECT frame before closing the socket. This reuses the same connect_to_maester() and send_frame() infrastructure from Phase 2, but introduces the

additional pattern of performing socket operations during the cleanup path rather than only during the interactive session.

- **Alliance timeout:** The server thread uses select() with a one-second timeout instead of blocking indefinitely on accept(). On each iteration, after select() returns, check_timeOuts() is called to scan the alliance array. Any alliance in ALLIANCE_PENDING or ALLIANCE_SENT state whose timestamp is more than 120 seconds old is automatically transitioned to ALLIANCE_REJECTED and a message is printed to the terminal informing the user. The NO_TIME sentinel value (defined as (time_t)-1) is used to indicate alliances that have been accepted or rejected and therefore should not be subject to timeout checks.

Phase 4

Phase 4 represents the final and most significant evolution of the Citadel System, bringing full concurrency into the picture through the introduction of Envoy processes. Where in previous phases all outgoing network operations were handled sequentially from the main thread, Phase 4 delegates every mission that requires network communication, such as pledges, product list queries, and trade orders into a bunch of independent child processes called Envoys. This allows the Maester to keep its interactive terminal fully responsive at all times while the Envoys carry out their missions in parallel. The architecture from Phase 3 is preserved in its entirety, so what changes is the execution model for outgoing actions and the coordination layer that connects the Maester with its Envoys.

Diagrams

The changes in Phase 4 are concentrated in the concurrency layer: the envoyConfig module, which was introduced as an empty placeholder in Phase 3, is now fully implemented and becomes the most central new component of this phase. A second background thread, the envoy listener, is added alongside the existing network listener, and the Maester's startup and shutdown sequences are updated accordingly.

System Architecture

The modular directory hierarchy from Phase 3 is unchanged. The additions and updates are mainly in the envoy-related architecture:

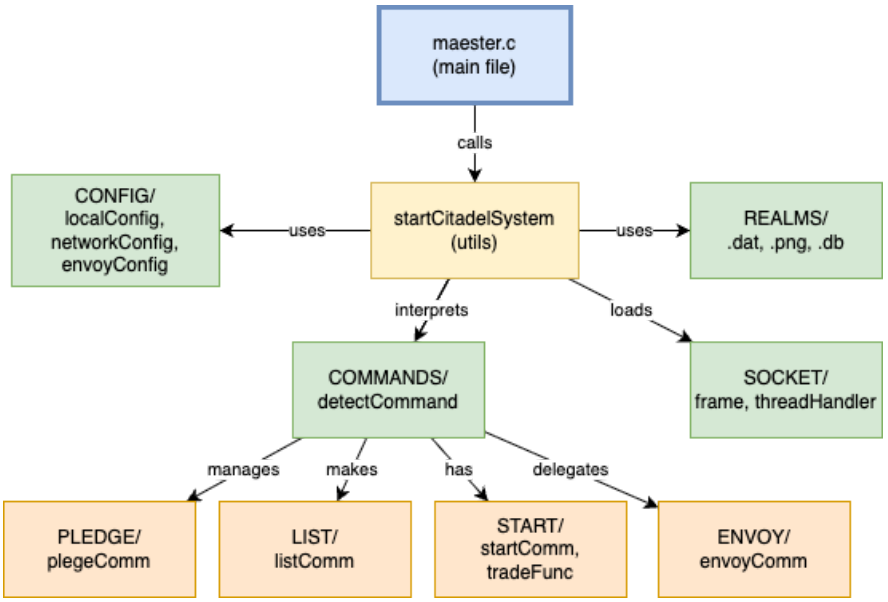


Figure 15: Diagram of Final System Architecture

The updated module's changes are described below:

- maester.c:

The entry point gains two new steps at startup. After binding the listening socket as in Phase 3, the program now also calls createEnvoys(), which forks one child process per Envoy defined in the .dat

file and wires each parent–child pair with two anonymous pipes. A second thread, envoy_thread(), is then created alongside the existing network_thread(). Both threads run in parallel with the interactive terminal for the entire duration of the program. At shutdown, after maesterDisconnection() sends disconnect notifications to all allies, an exit flag is set in the shared ServerComm structure, the listening socket is closed, and both threads are signalled with pthread_kill and joined with pthread_join before memory is freed.

- envoyConfig module (CONFIG/):

The central new module of Phase 4 and the one that was left empty in Phase 3. It provides createEnvoys(), which forks one child process per Envoy and connects each parent–child pair through two pipes; assignMission(), which packages the operation parameters into a Mission struct and writes it into the pipe of a free Envoy; envoyProcess(), which is the main event loop of each forked child and dispatches incoming missions to the appropriate handler depending on their type; envoy_thread(), which runs in the parent as a dedicated background thread monitoring all Envoy output pipes via select() and collecting results as they arrive; and check_envoyResponses(), which processes each completed mission and updates the Maester's alliance state, product cache, or terminal output accordingly. The four mission handler functions, which are execute_pledge_mission(), execute_pledge_response_mission(), execute_list_products_mission(), and execute_trade_mission(), each contain the full network logic that was previously called directly from the main thread, now running inside the child process.

- threadHandler module (SOCKET/) and networkConfig module (CONFIG/:

Unchanged from Phase 3. Incoming frames continue to be handled by the server thread as before, the Envoy layer concerns outgoing missions only. Since the implementation was already robust and correct, we didn’t feel the need to include new lines of code.

Program Execution

The program startup sequence gains two new steps compared to Phase 3. After reading configuration files and binding the listening socket, the program now also calls createEnvoys(), which forks one child process per Envoy and wires each one through two anonymous pipes. A second thread, the envoy listener, is then created alongside the existing network listener thread. Both threads run concurrently with the interactive terminal for the entire lifetime of the program.

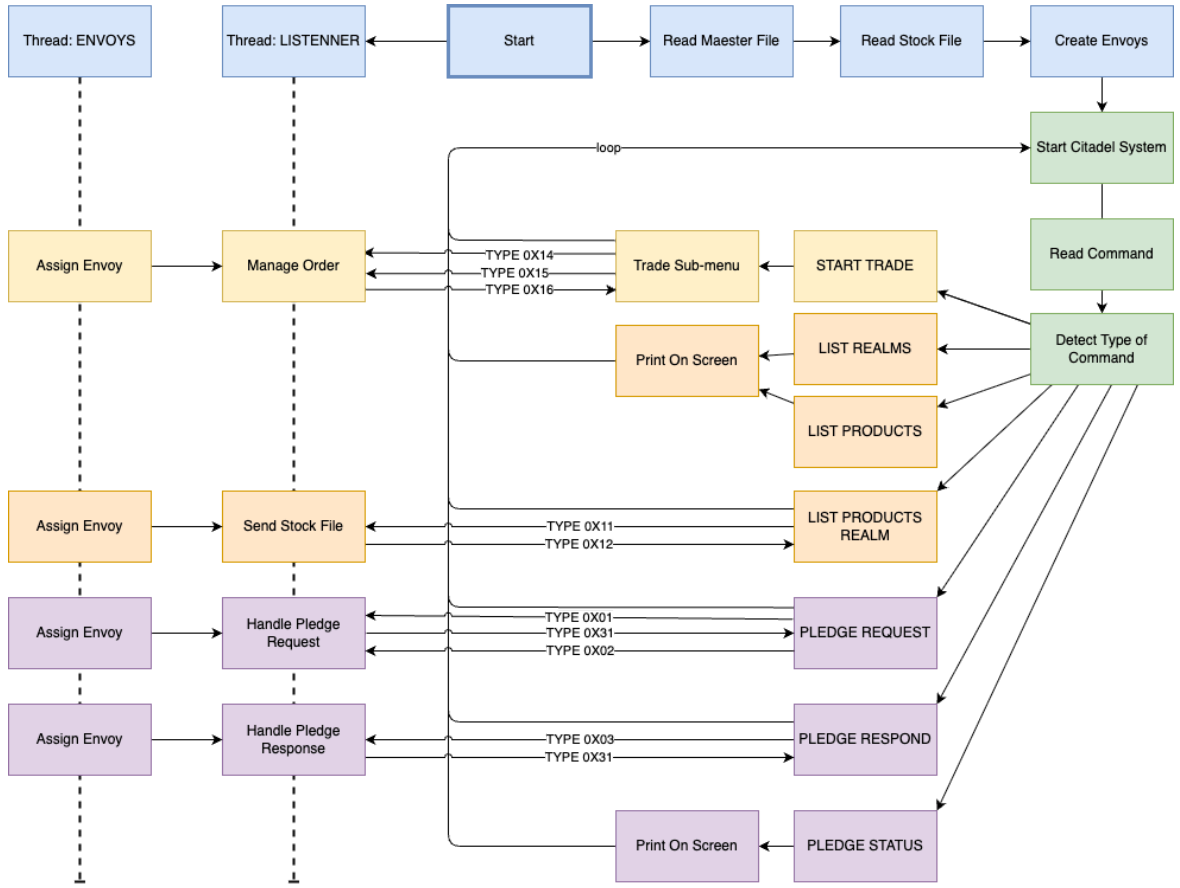


Figure 16: Diagram of Program Execution

At shutdown the sequence is also updated. After maesterDisconnection() notifies allied Maesters, the exit flag in the shared ServerComm structure is set so both threads can exit their loops cleanly. The listening socket is closed, then both the envoy listener and the network listener are signalled and joined before the Envoy pool is torn down. Each child process exits naturally when its pipe read end returns zero, so no explicit signals to the children are needed provided the parent closes the write ends in order.

Command Processing

Command processing follows the same multi-stage pipeline as Phase 3 without structural changes to the detectCommand() dispatcher. The difference in Phase 4 is entirely in what happens after command detection: instead of executing network operations directly from the main thread, commands that require outgoing communication now call assignMission(), which packages the relevant parameters into a Mission struct and writes it into the pipe of a free Envoy. The main thread then returns immediately to its read loop, leaving the Envoy to carry out the actual socket work independently.

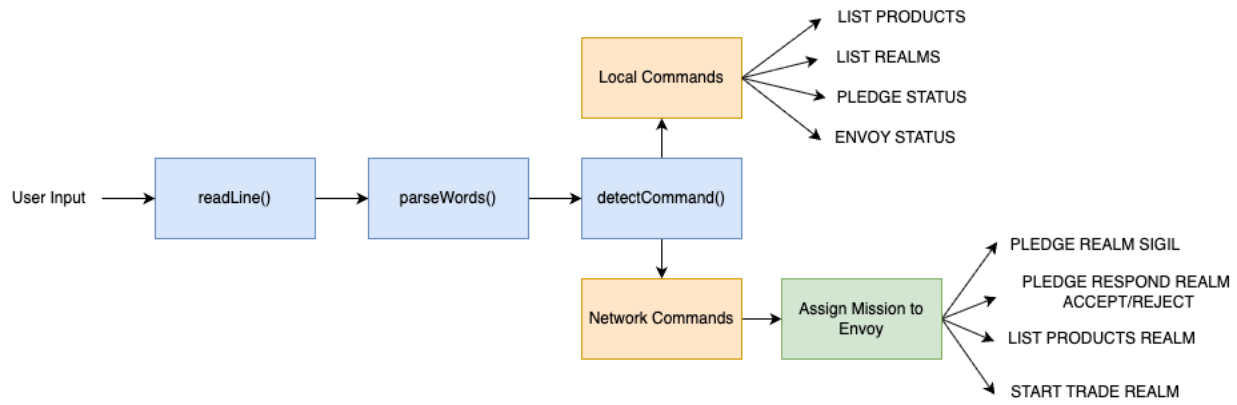


Figure 17: Diagram of Command Processing

Local commands such as LIST REALMS, PLEDGE STATUS, ENVOY STATUS, and EXIT continue to be handled synchronously in the main thread since they do not require any network access. The ENVOY STATUS command is new in this phase and prints the current state of each Envoy in the pool, showing ON_MISSION with the destination realm name for busy Envoys and FREE for idle ones.

Data Structures

There are a few new data structures that add up to the already existing structures from previous phases. Specifically, two new structures are added, and several existing ones gain new fields to support the Envoy layer.

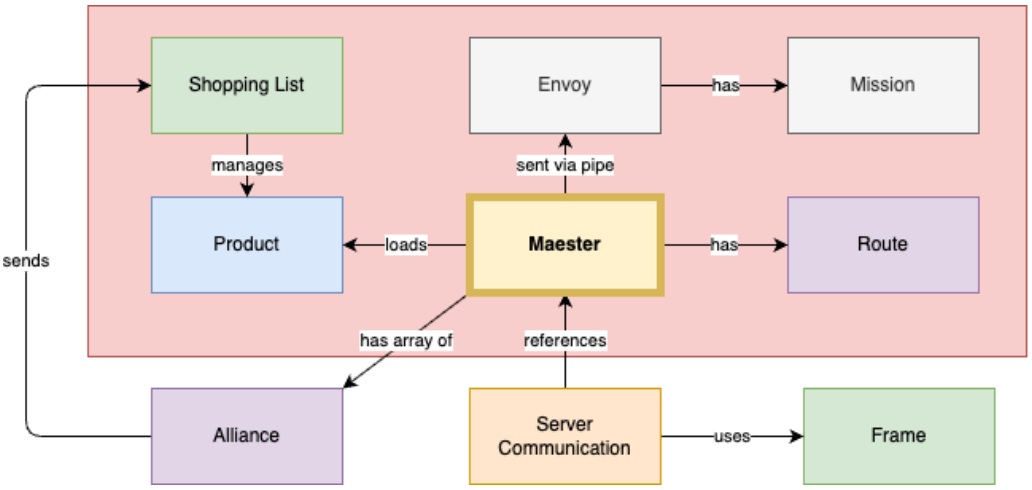


Figure 18: Diagram of Data Structures

- The Mission structure is the core new data type of Phase 4. It acts as the unit of work passed through the pipes between the Maester and an Envoy. It carries a type integer identifying which kind of mission to execute (MISSION_PLEDGE_REQUEST, MISSION_PLEDGE_RESPONSE, MISSION_LIST_PRODUCTS_REALM, or MISSION_START_TRADE), the origin and destination realm names alongside their IP addresses and port numbers, a fixed-size data buffer of 275 bytes for any additional payload such as a sigil file path or a shopping list filename, a success flag that the Envoy sets before writing the result back, and the PID of the Envoy process. Having all mission parameters

packed into a single fixed-size struct allows the parent to write the entire mission atomically with a single write() call and the child to read it just as simply, without any additional framing overhead on top of the pipe.

- The Envoy structure represents a single child process in the pool. It stores the two pipe file descriptor pairs (to_envoy and from_envoy), the child's PID obtained from fork(), a boolean flag isFree indicating whether the Envoy is available for a new mission, an integer mission field recording what type of mission it is currently running (used by the ENVOY STATUS display), a dynamically allocated string destination holding the name of the realm the Envoy is currently serving, and a sequential id starting from 1 used in user-facing messages such as "Pledge sent to TheVale. Envoy 1 is on its way."

- The Maester structure gains two new fields in Phase 4: ptEnvoys, a dynamically allocated array of Envoy structs created at startup by createEnvoys(), and nNumEnvoys, the integer already parsed from the .dat file since Phase 1 but now actually used to size and iterate over that array. A new mutex, mutex_envoys, is also added to protect concurrent access to the Envoy array between the main thread, which calls assignMission() and marks Envoys as busy, and the envoy listener thread, which marks them as free when their results arrive.

- The ServerComm structure, already present since Phase 2, gains two new fields: an exit integer flag and a mutex_exit mutex. These are used to signal both the network listener thread and the new envoy listener thread that the program is shutting down, so both can exit their select() loops cleanly without relying on signals alone.

System Resources

Phase 4 introduces two new OS resource categories on top of those used in Phase 3:

- Fork and anonymous pipes for Envoy concurrency: The most significant new resource category in Phase 4. During startup, createEnvoys() calls pipe() twice and then fork() once for each Envoy defined in the .dat file. This produces N child processes and 4N file descriptors (two pipe pairs per Envoy). The parent keeps the write end of to_envoy and the read end of from_envoy for each child, and closes the other two ends; the child does the inverse. Each Envoy child then runs its own blocking read loop inside envoyProcess(), waiting for Mission structs from the parent. This use of fork() and pipes is distinct from the fork() introduced in Phase 3 for MD5 computation: those were short-lived child processes spawned on demand, whereas the Envoy processes are long-lived workers that persist for the entire lifetime of the Maester. Proper cleanup requires the parent to close all pipe write ends before joining, so that each child's read() returns zero and the child exits naturally.

- A second background thread (envoy listener): Phase 3 introduced one background thread for the network listener. Phase 4 adds a second one, envoy_thread(), which runs a select() loop over all Envoy from_envoy read-end file descriptors simultaneously with a one-second timeout. When any Envoy completes a mission and writes its result back through the pipe, select() unblocks, the thread reads the result Mission, acquires mutex_envoys, calls check_envoyResponses() to update the appropriate state, and marks the Envoy as free again. This design means the interactive terminal never blocks on a network operation and multiple

Envoys can process missions concurrently. Coordination between this thread and the main thread is handled through mutex_envoys for Envoy state, mutex_alliances for alliance updates, and mutex_products for product cache updates, all three of which were already present in Phase 3 and continue to serve their original purpose. The thread is signalled with pthread_kill and joined with pthread_join at shutdown, after the exit flag has been set, in the same way as the network listener thread.

Optionals Implemented

For this whole project, we did not identify any functionality explicitly defined as optional in the statement. Therefore, we have implemented all the required features described for Phase 3. If any components are considered optional, they have been fully implemented as part of our solution.

Observed Problems & Solutions

A number of technological issues regarding data management, signal handling, and command processing arose throughout the system's development of Phase 1. The most pertinent problems are discussed in this part along with the remedies that were put in place to address them.

Phase 1

1. Incorrect quantity accumulation in the shopping list

Managing the shopping list during a trading session was one of the first issues found during testing. Quantities were not always handled correctly when adding products more than once. For instance, the shopping list should have a total of 29 units if a user entered 15 units of a product and then added 14 more. Nevertheless, the original method occasionally changed the existing quantity improperly or stored the quantities as distinct entries.

This problem resulted from adding a new element without adequately verifying if the product was already on the shopping list. Because of this, the program sometimes made multiple entries rather than updating the one that already existed.

A search mechanism was incorporated within the trade module to address this issue. The software now determines whether a product is already on the shopping list before adding it. The quantity is updated by adding the new value to the current one if the product is located. A new shopping list entry is made if it doesn't already exist.

2. Handling program interruption with CTRL+C

Using CTRL+C (SIGINT) to provide signal handling for program interruption presented another difficulty. First, if the program is interrupted, the system may abruptly shut down without releasing allotted RAM or appropriately finishing the execution sequence.

Cleaning up inside the signal handler is a typical strategy. Nevertheless, it is risky and may result in unclear behavior to carry out complicated operations like memory deallocation inside a signal handler. A safer procedure was put in place to deal with this problem. The signal handler merely uses the close(0) system function to close the standard input file descriptor rather than carrying out cleanup inside. The primary command-reading loop recognizes the end of input by terminating the input stream.

3. Input parsing reliability

During development, processing user commands presented another challenge. Before the system can understand commands, they must be divided into separate words since they are read as raw strings from standard input.

When handling extra spaces or unusual formatting in the user input, early implementations of the parsing algorithm occasionally provided wrong results. This might result in directives, even when they were legitimate, being misunderstood or denied.

A specific parsing utility function was put in place to increase reliability. The input line is tokenized by this function into distinct words, which are then stored in an array of strings. All command detection routines receive input in a uniform format thanks to the standardization of the parsing procedure.

4. Memory management considerations

Careful memory management was required to avoid memory leaks because the system uses dynamic memory allocation to store configuration data, product lists, and shopping lists. Some allocated memory blocks were not freed when the program ended during early testing, especially when leaving specific execution routes.

Phase 2

1. Server thread and main thread exit coordination

When the user typed EXIT or pressed Ctrl+C, the main loop would end and freeMaester would be called while the server thread was still blocked inside accept. This caused the server thread to access freed memory.

The solution was to close the listening socket from the main thread before joining. A blocked accept call returns immediately with an error when its socket is closed, causing the server thread to detect the error and exit its loop cleanly. pthread_join then guarantees all server-thread resources are released before the process ends.

2. Checksum validation failures due to byte-order inconsistency

During initial testing, frames sent from one Maester were rejected by another due to checksum mismatches. The root cause was that the checksum field was being written to the frame in host byte order on the sender and read in host byte order on the receiver, but validate_checksum was calling ntohs before comparing, creating an asymmetry.

The fix was to ensure that build_frame always stores the checksum with htons, and that validate_checksum always reads it with ntohs before comparing against the locally recomputed value. Zeroing the checksum field before recomputing is also required; this is done explicitly in validate_checksum before calling calculate_checksum.

3. Multi-hop routing with wildcard IP

Realms whose .dat file lists a route as *.*.*.* cannot be reached directly. The initial implementation skipped the wildcard check in pledge_request, causing connection failures when a destination was only reachable through the DEFAULT route.

The fix was to add an explicit check in both pledgeRequest (client side) and pledge_request (server-side hop forwarding): if the looked-up IP is *.*.*.*, the function falls back to the DEFAULT route for the actual socket connection while preserving the original destination in the frame's dest field so that the next hop can continue routing correctly.

4. Alliance array dynamic growth

The Alliance array in the Maester structure needed to grow as new alliances were created at runtime, a situation that does not arise for the Routes array which is fully known at startup. Early versions crashed when a second alliance was created because realloc was not used.

The createAlliance utility function was introduced to handle this correctly: it calls realloc on ptAlliances, increments nNnumAlliances, and initialises all fields of the new entry to ensure independent ownership of those strings.

Phase 3

1. File transfer fragmentation and reassembly errors

During the implementation of file transfer (sigils and trade files), one of the main issues encountered was related to handling fragmented data. Since files must be split into multiple frames (TYPE 0x02, 0x13), early versions of the system did not correctly reconstruct the original file on the receiver side. This led to incomplete or corrupted files.

To solve this, a controlled buffering mechanism was introduced. The receiver now dynamically allocates a buffer based on the file size specified in the header and appends each incoming fragment sequentially. A counter is maintained to track the total received bytes, ensuring the transfer is only considered complete when the expected size is reached.

2. MD5 verification inconsistencies

Another major issue appeared when validating transferred files using the md5sum command. Even when files appeared correct, the computed MD5 checksum sometimes did not match the original one sent in the header. This was mainly due to subtle differences in how the file was reconstructed, such as extra bytes, missing padding, or incorrect termination of buffers.

The solution was to strictly ensure that: only the exact number of bytes specified in the header is written to the file, no additional padding or null characters are included in the final file, and the MD5 is computed using the exact reconstructed file via the system command. After enforcing these constraints, MD5 validation became consistent and reliable.

3. Stock update consistency after trade

Another issue arose when updating the inventory after processing an order. In early implementations, stock changes were applied in memory but not always correctly persisted to the stock.db file. This caused discrepancies between the runtime state and the stored data after restarting the program.

The fix involved enforcing immediate persistence: after every successful trade, the updated inventory is rewritten to the binary file and proper file handling (open, write, close) ensures durability.

4. Memory management for large transfers

Handling large files required dynamic memory allocation for buffers. During early testing, some buffers were not properly freed after transfer completion or failure, leading to memory leaks.

To resolve this we made all dynamically allocated buffers freed after use, also error paths (e.g., failed transfer, MD5 mismatch) were explicitly released allocated memory.

Phase 4

During the implementation of Phase 4, several challenges appeared regarding process synchronization, concurrent task delegation, and inter-process communication between the Maester and the Envoys. Since Envoys were implemented as fully independent processes using forks, special care was required to guarantee correct coordination and avoid race conditions or resource conflicts.

1. Envoy allocation conflicts during concurrent missions

One of the first issues detected during testing appeared when multiple commands were issued quickly from the terminal. In some situations, two different missions were assigned to the same Envoy process simultaneously, causing duplicated operations or inconsistent mission states.

The problem originated from the fact that the Maester accessed the Envoy status table concurrently without guaranteeing mutual exclusion while checking for available Envoys. As a result, two different execution paths could detect the same Envoy as FREE before its state was updated.

To solve this issue, a synchronization mechanism was introduced around the shared Envoy state structure. Before assigning a mission, the Maester now locks access to the Envoy table, verifies the status atomically, updates the Envoy state to ON_MISSION, and only then releases the lock. This guarantees that each mission is assigned exclusively to one Envoy process.

2. Pipe communication blocking between Maester and Envoys

Communication between the Maester and the Envoys was implemented using pipes, since the statement explicitly forbids the use of sockets or secondary memory for internal communication. During early testing, some Envoy processes became permanently blocked while waiting for data from the Maester.

This occurred because certain pipe descriptors were not properly closed after fork creation. As a consequence, the reading side of the pipe never received an EOF condition, causing blocking reads even when the sender process had already terminated.

The solution consisted of carefully managing all pipe file descriptors immediately after each fork. Every process now closes all unused pipe ends, ensuring that only the intended reader and writer retain access to the communication channel. This allowed blocking reads to behave correctly and prevented deadlocks between processes.

3. Mission timeout handling in independent Envoys

Managing timeouts became more complex once missions were delegated to independent Envoy processes. In early versions, if a remote realm did not respond, the responsible Envoy could remain

blocked indefinitely waiting for network data, effectively reducing the number of available Envoys over time.

The solution was to integrate timeout handling directly inside the Envoy communication logic. Socket operations now use configurable time limits, allowing Envoys to detect unresponsive peers. When a timeout occurs, the Envoy reports the mission failure back to the Maester, releases all allocated resources, and terminates cleanly.

4. Envoy termination during Maester shutdown

A final challenge appeared when the Maester process terminated while Envoys were still executing active missions. Initially, some Envoys continued running independently after the Maester exited, leaving orphan processes executing in the background.

To address this problem, the EXIT sequence was redesigned so that the Maester first notifies all active Envoys that the system is shutting down. The Maester then waits for every Envoy to terminate correctly before releasing shared resources and ending execution itself. This guarantees a controlled and clean shutdown of the entire system.

Temporal Estimation

This section provides an estimation of the time invested in the development of the project by each member of the team. The work was divided into several categories that reflect the different phases of the development process: research, design, implementation, testing, and documentation.

The following estimations do not include the time spent during the official laboratory sessions and only account for the work carried out independently by the students.

Phase 1

Sara

The total estimated time invested by Sara in the development of the project was approximately **24 hours**. The distribution of time among the different tasks is described below.

Research: 4h

During this time, documentation pertaining to C programming concepts, such as dynamic memory management, file handling, and signal handling, that were necessary for the project was reviewed. Finding examples and comprehending the system calls utilized during the implementation also took more effort.

Design: 4h

Sara helped define the program's modular architecture during the design stage. This involved arranging how the various parts would work together and talking about the division of the system into modules (config, commands, trade, and utils). Outlining the structure of the data types used in the project also took time.

Implementation: 10h

Writing and integrating the program code was part of the implementation phase. Managing data structures, integrating the various system modules, and developing command processing routines were among the tasks.

Testing: 3h

To ensure proper functioning, the program was tested using various input circumstances. Testing the command system, trade session activities, and edge problems like improper input or repeated commands received special attention.

Documentation: 3h

Writing the project report, recording the design choices, and adding internal comments to the code to clarify the operation of important modules and functions took up this time.

Oriol

The total estimated time invested by Oriol in the development of the project was approximately **26 hours**. The time distribution across the project phases is described below.

Research: 3h

Oriol reviewed the project's technical specifications and looked into practical C programming methods for processing structured data, parsing input, and reading files.

Design: 4h

Oriol took part in developing the program's overall architecture and spoke about how the modules would interact with one another during the design phase. Planning the trade session process and command structure was another aspect of this step.

Implementation: 13h

The implementation of the program logic took up the majority of the development time. This involved developing the trade session feature, integrating the command detection system, and writing multiple modules.

Testing: 3h

Testing involved confirming the program's operation using various command sequences and confirming that product lists, trade sessions, and error instances were handled correctly.

Documentation: 3h

Oriol also helped with the project documentation preparation and code review, making sure that pertinent areas were adequately annotated.

Phase 2

Sara

The total estimated time invested by Sara in Phase 2 was approximately **28 hours**, distributed as follows.

Research: 5h.

Sara reviewed the socket programming, the pthread API, and TCP connection lifecycle management, and all the related work with fixing the feedback relative to Phase 1.

Design: 5h.

The redesign of the module structure into subdirectories, the definition of the binary frame protocol, and the alliance state machine were all discussed and sketched out during this stage.

Implementation: 12h.

Sara led the implementation of the frame module (serialisation, checksum, send/receive), the network configuration module, and the pledge command on the sending side.

Testing: 3h.

Testing involved running multiple Maester instances simultaneously on the same machine with different ports and verifying that alliance requests, responses, and product list queries behaved correctly across them.

Documentation: 3h.

Writing this report section and annotating the new modules with function-level comments.

Oriol

The total estimated time invested by Oriol in Phase 2 was approximately **26 hours**, distributed as follows.

Research: 4h.

Oriol focused on the pthread threading model, socket options, and binary protocol design considerations.

Design: 5h.

Oriol contributed to the design of the server thread architecture and the routing fallback logic, and outlined the command dispatch changes needed in detectCommand.

Implementation: 11h.

Oriol implemented the server thread, the thread handler functions for incoming frames, the pledge respond and pledge status commands, and the extended LIST PRODUCTS network flow.

Testing: 3h.

Testing included verifying multi-hop forwarding, timeout behaviour, and edge cases such as pledging a realm that is already an ally or already pending.

Documentation: 3h.

Oriol contributed to this report section and reviewed the code for consistent annotation.

Phase 3

Sara

The total estimated time invested by Sara in Phase 3 was approximately **20 hours**, distributed as follows.

Research: 1h

Sara reviewed file transfer mechanisms in C, focusing on handling binary data, fragmentation of files into frames, and reconstruction on the receiver side. Additional time was spent understanding the use of external system commands such as md5sum for file integrity verification and studying safe file I/O practices.

Design: 3h

During this phase, Sara contributed to the design of the file transfer workflow, including the separation between header and data frames, and how these integrate with the existing protocol. The structure for buffering incoming data, validating transfers, and updating the inventory consistently was also planned.

Implementation: 11h

Sara worked on implementing the file transfer logic, including sending and receiving fragmented data (TYPE 0x02 and 0x13), reconstructing files, and integrating MD5 verification. She also contributed to updating the inventory system to reflect accepted trade operations and ensuring persistence in stock.db.

Testing: 2h

Testing focused on validating full file transfers between multiple Maesters, including both direct and multi-hop scenarios. Special attention was given to detecting corrupted transfers, verifying MD5 correctness, and ensuring correct behavior when stock was insufficient or products did not exist.

Documentation: 3h

Time was dedicated to updating the report with the new design decisions, describing the file transfer protocol, and documenting the implemented solutions and encountered issues.

Oriol

The total estimated time invested by Oriol in Phase 3 was approximately **16 hours**, distributed as follows.

Research: 1h

Oriol focused on studying efficient buffer management, file handling in binary mode, and reliability considerations in network-based file transfer. He also reviewed methods for integrating external commands like md5sum within the program.

Design: 1h

Oriol contributed to the design of the data flow for trade and alliance file transfers, including how intermediate nodes should handle frames transparently. He also helped define how stock validation and order rejection should be handled.

Implementation: 12h

Oriol implemented the receiving side of file transfers, including buffering, reconstruction, and MD5 validation. He also worked on processing trade orders, validating stock availability, and ensuring correct updates to the inventory file. Additional work included refining routing behavior for file transfer frames.

Testing: 1h

Testing included multi-node scenarios with file transfers passing through intermediate hops, verifying that data integrity was preserved. Edge cases such as incomplete transfers, invalid MD5, and insufficient stock were also tested.

Documentation: 1h

Oriol contributed to documenting the implementation details of Phase 3, reviewing the report, and ensuring consistency across all sections.

Phase 4

Sara

The total estimated time invested by Sara in Phase 4 was approximately **20 hours**, distributed as follows.

Research: 4h

Sara focused on understanding process concurrency in depth, particularly the use of fork and pipes required for inter-process communication between the Maester and Envoys. Additional time was spent studying thread safety and synchronization techniques, which finally chose mutexes, since Phase 4 introduced strict constraints on shared resource access and required careful coordination between multiple concurrent execution flows.

Design: 5h

During the design stage, Sara contributed to redefining the system architecture to incorporate the Envoy model. This included planning how missions (Pledge and Trade operations) would be delegated from the Maester to available Envoys, and how Envoy state would be tracked. The structure of shared memory segments and synchronization mechanisms was also defined, ensuring that no sockets or secondary storage were used for Maester–Envoy communication as required by the specification.

Implementation: 7h

Sara worked on implementing the Envoy lifecycle, including creation, initialization, and task assignment logic. This involved integrating fork calls for Envoy creation, assigning unique identifiers and PIDs, and implementing mission dispatching from the Maester. Additional work included developing synchronization mechanisms for shared resources, ensuring that multiple Envoys could operate concurrently without race conditions while accessing shared mission queues and status data.

Testing: 2h

Testing focused on validating concurrent execution of multiple Envoys under heavy load scenarios. Different combinations of simultaneous Pledge and Trade requests were executed to verify correct task distribution, proper handling of shared resources, and absence of deadlocks. Special attention was given to ensuring that Envoys correctly reported task completion and that Maester state remained consistent under concurrency.

Documentation: 2h

Sara updated the report to reflect the introduction of the Envoy-based architecture, documenting design decisions, concurrency mechanisms, and synchronization strategies used in Phase 4. Code comments were also extended to clarify process lifecycle management and shared resource protection.

Oriol

The total estimated time invested by Oriol in Phase 4 was approximately **24 hours**, distributed as follows.

Research: 4h

Oriol studied advanced inter-process communication techniques, focusing on process creation using fork and synchronization mutexes. He also reviewed potential race condition scenarios and explored best practices for safe concurrent task assignment between multiple child processes (Envoys).

Design: 5h

Oriol contributed to designing the Envoy task distribution system, defining how the Maester assigns missions dynamically to available Envoys. He also helped design the shared state structure used to track Envoy availability, active missions, and completion status. Special emphasis was placed on ensuring compliance with the restriction of no sockets or disk-based communication between Maester and Envoys.

Implementation: 11h

Oriol implemented the core concurrency logic, including Envoy spawning using fork, signal handling for Envoy lifecycle management, and the task queue system used to distribute missions. He also implemented mutex-protected shared structures to avoid race conditions and ensured correct synchronization between Envoys updating mission states and the Maester reading them.

Testing: 3h

Testing involved stress scenarios with multiple simultaneous missions, verifying correct Envoy allocation, task execution integrity, and system stability under concurrent load. Oriol also tested edge cases such as all Envoys being busy, ensuring that additional missions were queued or correctly handled without system crashes.

Documentation: 1h

Oriol contributed to the final Phase 4 report section, ensuring consistency with previous phases and documenting the concurrency model, Envoy lifecycle, and synchronization approach used in the final system.

Total Time

Here is a resume-table showcasing the total hours inverted by each partner member:

	SARA	**ORIOL**
Phase 1	24h	26h
Phase 2	28h	26h
Phase 3	20h	16h
Phase 4	20h	24h
TOTAL	**92h**	**92h**

Conclusions & Proposals for Improvement

The development of the Citadel System has progressively transformed a local, single-process application into a fully distributed, concurrent communication network capable of handling complex interactions between independent realms. Across its four phases, the system evolved in a structured and incremental manner, with each stage building directly upon the previous one to ensure scalability, modularity, and robustness.

In Phase 1, the foundation of the system was established through the implementation of the Maester process. This included the design of the core architecture, configuration parsing, and the interactive command terminal. At this stage, the focus was placed on structuring the program into modular components capable of managing user commands, loading configuration data, and maintaining persistent inventory information. This phase provided the essential internal structure required for all subsequent functionality.

In Phase 2, the system was extended into a distributed environment through the introduction of inter-Maester communication. Using socket-based networking and a hop-based routing mechanism, the Citadel System became capable of transmitting messages across multiple realms. This phase introduced the foundation of the communication protocol, including message framing, routing decisions, and basic alliance formation. As a result, the system transitioned from a standalone application into a multi-node network, where each Maester could act both as a sender and an intermediate relay.

In Phase 3, the system was significantly expanded with the implementation of full file transfer capabilities and complete trade functionality. The introduction of fragmented transmission, MD5-based integrity verification, and persistent stock management enabled the secure exchange of real data between allied realms. At this point, alliances were no longer purely symbolic, as they supported meaningful economic interaction through product lists and trade orders. The system demonstrated reliable end-to-end communication with validation and state consistency across distributed nodes.

Finally, in Phase 4, the concurrency model of the system was enhanced through the introduction of the Council of Envoys. This allowed each Maester to handle multiple simultaneous missions by delegating tasks to independent Envoy processes. This design eliminated previous limitations on single-task execution and significantly improved system concurrency. The Maester maintained control over routing and supervision, while Envoys managed individual communication flows, enabling efficient parallel execution of alliance and trade operations.

Overall, the Citadel System successfully achieves its objective of simulating a robust distributed network inspired by operating system principles. It integrates key concepts such as process management, inter-process communication, concurrency, routing algorithms, and data integrity verification into a cohesive architecture. Each phase contributed essential functionality, culminating in a system capable of reliable communication, secure data exchange, and scalable concurrent task execution across multiple independent nodes.

Although the Citadel System achieves a complete and functional distributed architecture, just to be a little bit picky about our project, there are still several aspects that could be refined to further enhance

its performance, scalability, and robustness. The concurrency model could be strengthened by introducing finer-grained synchronization techniques or alternative thread-safe data structures to minimise contention between Envoys when accessing shared resources. Error handling across the system could also be made more resilient, particularly in scenarios involving partial failures, delayed responses, or sudden disconnections, ensuring smoother recovery without disrupting ongoing operations. Finally, performance under high load could be improved through optimised resource management, including better scheduling of Envoy tasks and reduced overhead in communication handling.

2025-2026 Project's Theme

For us, it was easy to identify this year's topic since we're both big fans of it. This year's project's subject draws inspiration from the Game of Thrones realm, namely the academic establishment known as the Citadel. In the series, the Citadel serves as the epicenter of knowledge on the continent of Westeros, where academics known as Maesters conduct research, oversee data, and counsel various regional leaders.

This theme serves as a narrative framework for the project's various system components. A Maester, who serves as the central controller in charge of information management and trade operations coordination, represents the main program. Information about several worlds, which stand in for the areas where trade is possible, is available to the Maester.

Additionally, the system maintains a list of products that match the goods that can be traded. The program mimics the creation of a shopping list that will be delivered to a certain location when the user initiates a trade session with that realm. The Maester arranges requests for products before sending them, simulating a streamlined trading procedure.

This topic is also reflected in other project concepts. For instance, trading sessions mimic interactions between several fictitious world worlds, and the Citadel system serves as the primary environment in which commands are carried out.

Bibliography

[1] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1988.

[2] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco, CA, USA: No Starch Press, 2010.

[3] ISO/IEC, *ISO/IEC 9899:2018: Programming Languages — C*. Geneva, Switzerland: International Organization for Standardization, 2018.

[4] Linux Manual Pages Project, "Linux man-pages," [Online]. Available: https://man7.org/linux/man-pages/. Accessed: Mar. 3, 2026.

[5] GNU Project, "The GNU C Library Reference Manual," [Online]. Available: https://www.gnu.org/software/libc/manual/. Accessed: Mar. 2, 2026.

[6] Stack Overflow, "Stack Overflow – Where developers learn, share, and build careers," [Online]. Available: https://stackoverflow.com. Accessed: Mar. 7, 2026.

[7] W. R. Stevens, B. Fenner, and A. M. Rudoff, UNIX Network Programming, Volume 1: The Sockets Networking API, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003. [Online]. Available: https://www.pearson.com/en-us/subject-catalog/p/unix-network-programming-volume-1-the-sockets-networking-api/P200000000584

[8] W. R. Stevens and S. A. Rago, Advanced Programming in the UNIX Environment, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2013. [Online]. Available: https://www.pearson.com/en-us/subject-catalog/p/advanced-programming-in-the-unix-environment/P200000000655

[9] D. Comer, Internetworking with TCP/IP, Volume 1: Principles, Protocols, and Architecture, 6th ed. Boston, MA, USA: Pearson, 2014. [Online]. Available: https://www.pearson.com/en-us/subject-catalog/p/internetworking-with-tcp-ip-volume-1/P200000003214

[10] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2015. [Online]. Available: https://www.pearson.com/en-us/subject-catalog/p/modern-operating-systems/P200000003386

[11] A. S. Tanenbaum and M. van Steen, Distributed Systems: Principles and Paradigms, 2nd ed. Upper Saddle River, NJ, USA: Pearson, 2007. [Online]. Available: https://www.distributed-systems.net/index.php/books/distributed-systems-2nd-edition/

[12] J. Postel, "Transmission Control Protocol," RFC 793, Sep. 1981. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc793

[13] S. Peterson and M. Davie, Computer Networks: A Systems Approach, 5th ed. Boston, MA, USA: Morgan Kaufmann, 2011.

[14] R. Stevens, UNIX Network Programming, Vol. 2: Interprocess Communications, 2nd ed. Prentice Hall, 1999.

[15] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.